

Creating Slides

Abdul Saboor¹, Unknown Author²

May 18, 2026

¹My University is somewhere in the middle of nowhere

²Their University is somewhere in the middle of nowhere

Right click (or click on footer) to open context menu and click on ⓘ icon for instructions.

≡ Show Code

Contents

1. **Introduction**
2. Adding informative TOC
3. Plotting and DataFrame
4. Simple Animations with Frames
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides

Introduction

To see how commands work, use `slides.docs()` to see the documentation. Here we will focus on using some of that functionality to create slides.



Info

This slide was built purely from markdown, so you can create a variable `test` to overwrite this →

Exception: Could not resolve '%{test}':

NameError: name 'test' is not defined

You can update this variable by `'Slides[int,|list|slice].vars.update'` or by defining it in a notebook if `'Auto Rebuild'` is enabled.

Version: 7.0.0

Contents

1. Introduction
2. **Adding informative TOC**
3. Plotting and DataFrame
4. Simple Animations with Frames
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides

This is summary for current section created using block syntax of toc. See `Slides.xml.syntax` for details.

- Item 1
- Item 2

$$E = mc^2$$



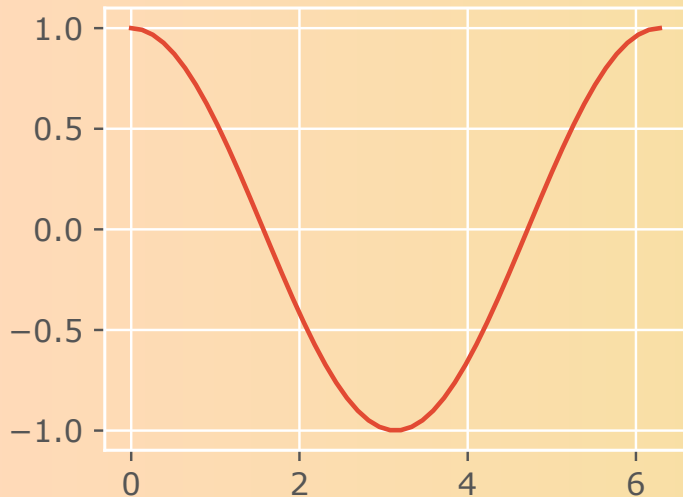
Tip

Above `btn` variable can be updated later via `Slides[number,].vars.update` method.

Markdown

```
1 section`Adding informative TOC`
2 ````columns .block-blue
3 toc[True]`### Contents`
4 ...
```

Plotting with Matplotlib



Python

```
1 slides.css(bg1 = 'linear-gradient(to right, #FFDAB9  
2  
3 import numpy as np, matplotlib.pyplot as plt  
4 plt.rcParams['svg.fonttype'] = 'none' # Global setti  
5 x = np.linspace(0,2*np.pi)  
6 with plt.style.context('ggplot'):  
7     fig, ax = plt.subplots(figsize=(3.4,2.6))  
8     _ = ax.plot(x,np.cos(x))  
9 slides.write(ax, s.focus([0,3,4]))  
10 slides.write('Double click on the plot to focus on i
```

Double click on the plot to focus on it!

Writing Pandas DataFrame

	sepal_length	sepal_width	petal_length	petal_width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

Python

```
1 try:
2     import ipywidgets as ipw
3     import plotly.graph_objects as go
4
5     fig = slides.patched_plotly(go.FigureWidget()) # prefer Widget for interactivity and correct display
6     fig.add_trace(go.Bar(y=[1,5,8,9], customdata=["A", "B"])))
7
```

Writing Plotly Figure

Install `plotly` to view output

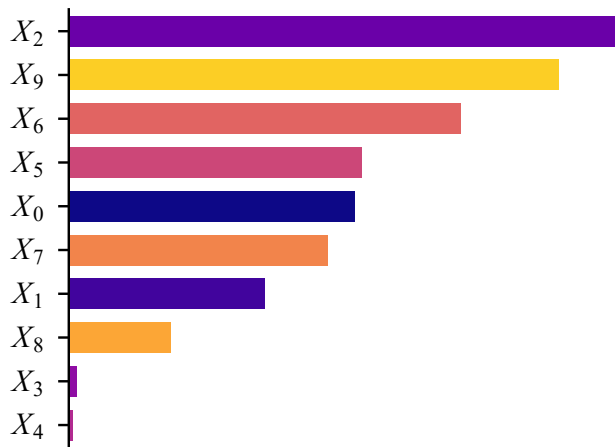
Python

```
1 try:
2     import ipywidgets as ipw
3     import plotly.graph_objects as go
4
5     fig = slides.patched_plotly(go.FigureWidget()) # prefer Widget for interactivity and correct display
6     fig.add_trace(go.Bar(y=[1,5,8,9], customdata=["A", "B"])))
7
8     # We have clicked and selected traits on patched plotly
9     html = ipw.HTML()
10
11     def observe_click(change):
12         html.value = "<br/>".join(f" {k} = {v}" for k, v in change['new'].items())
13
14     fig.observe(observe_click, names='clicked')
15     box = ipw.HBox([fig, html])
16
17 except:
```

Refreshable Content

Use refresh button below to update plot! See `race_plot` function at end of slides.

Race Plot



A Silly Plot

 Refresh Plot

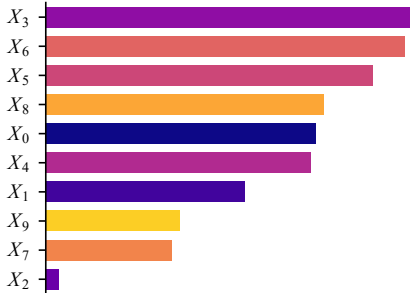
Python

```
1 slides.write('''
2     ## Refreshable Content
3     Use refresh button below to update plot!
4     See alert`race_plot` function at end of slides.
5     ''')
6
7 def display_plot(btn): return race_plot().display()
8
9 slides.write(
10     slides.dl.interactive(display_plot, btn = slides
11     rslide.get_source()
12 ) # Only first columns will update
```

Animations with Widgets

anim    12 Animation Frame: 12

Race Plot



A Silly Plot

Python

```
1 slides.write('## Animations with Widgets')
2 anim = slides.AnimationSlider(nframes=20, interval=1500, c
3 css = {'grid-template-columns': '1fr 2fr', '.out-main': {
4
5 @slides.dl.interact(post_init = lambda self: self.set_css(
6 def _(html, source, anim):
7     html.value = race_plot().value
8     print(f'Animation Frame: {anim}') # goes to output area
```

Rich Content ListWidget

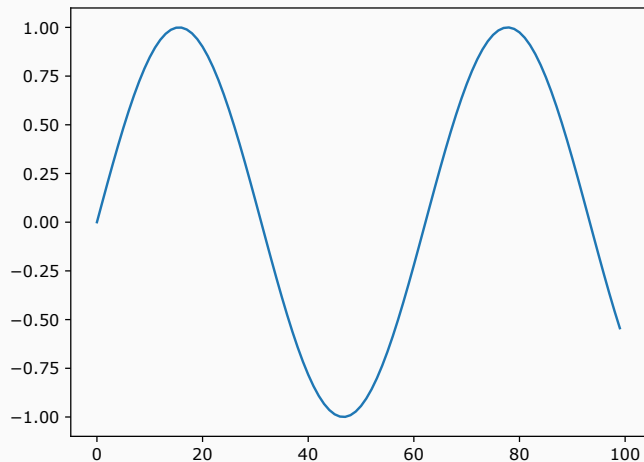
Execute a code block

```
lambda: print(np.random.random((10,2))),
```

```
lambda: plt.plot(np.random.random((10,2))),
```

```
def plot_sine():
```

```
    plt.plot(np.sin(np.linspace(0,10,100)))
```



Python

```
1 @slides.build_
2 def _(s):
3     import numpy as np
4     import matplotlib.pyplot as plt
5
6     def plot_sine():
7         plt.plot(np.sin(np.linspace(0,10,100)))
8
9     lw = slides.ListWidget(description='Execute a code block',
10                             options = [
11                                 lambda: print(np.random.random((10,2))),
12                                 lambda: plt.plot(np.random.random((10,2))),
13                                 plot_sine,
14                             ], transform = lambda value: slides.code(value))
15
16
17 def run(c):
18     if callable(c): c() # avoid None value when
19     plt.show()
20
```

Contents

1. Introduction
2. Adding informative TOC
3. Plotting and DataFrame
- 4. Simple Animations with Frames**
5. Controlling Content on Frames
6. Custom Objects Serilaization
7. Code to Generate Slides

Animating Matplotlib!

[→ Skip All Next Frames](#)

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

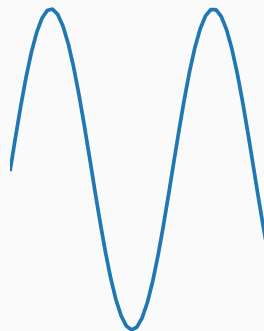
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 11$$



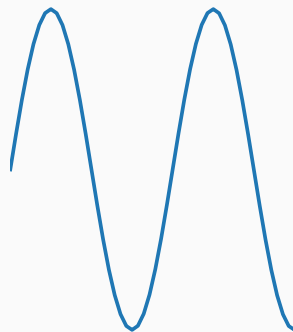
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 12$$



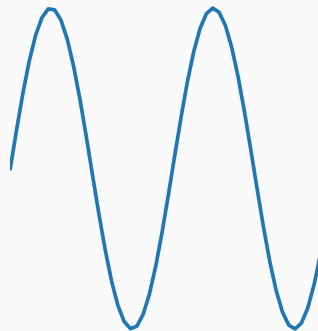
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 13$$



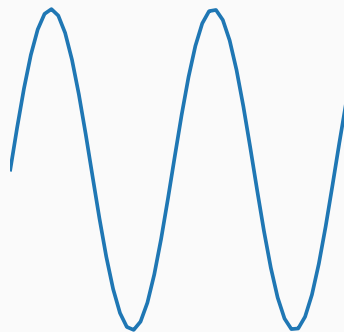
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 14$$



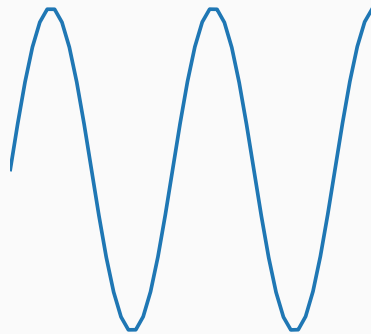
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 15$$



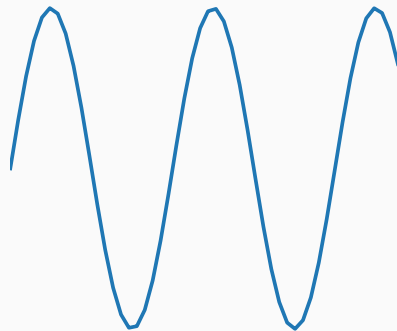
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 16$$



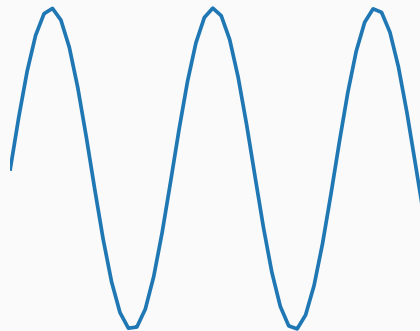
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 17$$



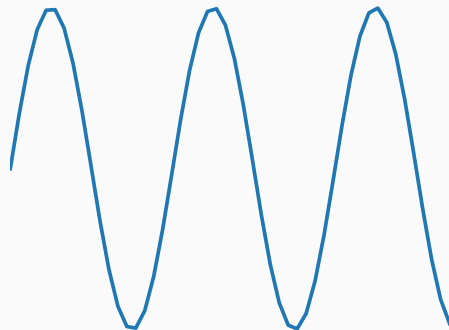
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 18$$



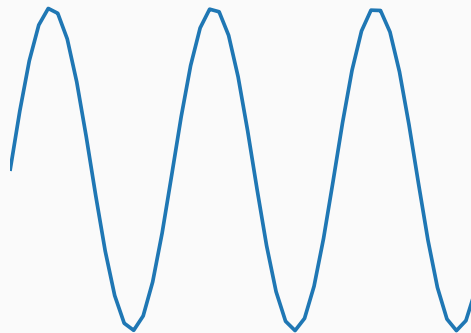
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 19$$



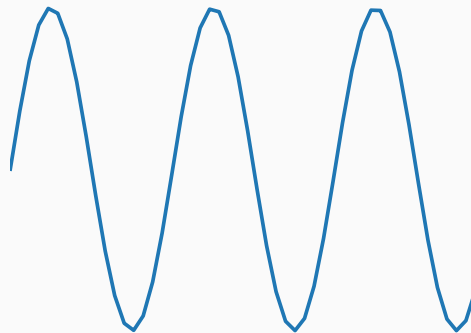
Animating Matplotlib!

→ Skip All Next Frames

Python

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 plots = slides.group(snapshots=True) # create empty snapshots-l
5 for idx in range(10,19):
6     _, ax = plt.subplots(figsize=(3.4,2.6))
7     x = np.linspace(0,idx,50)
8     ax.plot(x,np.sin(x))
9     ax.set_title(rf'$f(x)=\sin(x)$, 0 < x < {idx+1}$')
10    ax.set_xlim([0,18])
11    ax.set_axis_off()
12    plots.append(ax)
```

$$f(x) = \sin(x), 0 < x < 19$$



Tip

Unlike `interact/interactive`, this animation is based on slide frames, all of which are exported to HTML.

Contents

1. Introduction
2. Adding informative TOC
3. Plotting and DataFrame
4. Simple Animations with Frames
- 5. Controlling Content on Frames**
6. Custom Objects Serilaization
7. Code to Generate Slides

Frames with Snapshots

Python

```
1 slides.write('# Frames with Snapshots yoffset`0`')
2 s.get_source().focus([2,3,4]).display()
3 icons = {1:'edit', 2:'gear', 3:'download', 4:'trash'} # just for fun, can be any marker or None
4 slides.pause(isolate=True) # isolate to split from previous content
5 slides.write(slides.group(boxes,
6     points=True, snapshots=True,
7     marker=lambda idx:f"fa['red']`{icons[idx]}`",
8     css_class='anim-group anim-slide-left', # per item animation via parent group
9 )) # snapshots reveal rows in isolation
```

Frames with Snapshots

Python

```
1 slides.write('# Frames with Snapshots yoffset`0`')
2 s.get_source().focus([2,3,4]).display()
3 icons = {1:'edit', 2:'gear', 3:'download', 4:'trash'} # just for fun, can be any marker or None
4 slides.pause(isolate=True) # isolate to split from previous content
5 slides.write(slides.group(boxes,
6     points=True, snapshots=True,
7     marker=lambda idx:f"fa['red']`{icons[idx]}`",
8     css_class='anim-group anim-slide-left', # per item animation via parent group
9 )) # snapshots reveal rows in isolation
```



1

Frames with Snapshots

Python

```
1 slides.write('# Frames with Snapshots yoffset`0`')
2 s.get_source().focus([2,3,4]).display()
3 icons = {1:'edit', 2:'gear', 3:'download', 4:'trash'} # just for fun, can be any marker or None
4 slides.pause(isolate=True) # isolate to split from previous content
5 slides.write(slides.group(boxes,
6     points=True, snapshots=True,
7     marker=lambda idx:f"fa['red']`{icons[idx]}`",
8     css_class='anim-group anim-slide-left', # per item animation via parent group
9 )) # snapshots reveal rows in isolation
```



2

Frames with Snapshots

Python

```
1 slides.write('# Frames with Snapshots yoffset`0`')
2 s.get_source().focus([2,3,4]).display()
3 icons = {1:'edit', 2:'gear', 3:'download', 4:'trash'} # just for fun, can be any marker or None
4 slides.pause(isolate=True) # isolate to split from previous content
5 slides.write(slides.group(boxes,
6     points=True, snapshots=True,
7     marker=lambda idx:f"fa['red']`{icons[idx]}`",
8     css_class='anim-group anim-slide-left', # per item animation via parent group
9 )) # snapshots reveal rows in isolation
```



3

Frames with Snapshots

Python

```
1 slides.write('# Frames with Snapshots yoffset`0`')
2 s.get_source().focus([2,3,4]).display()
3 icons = {1:'edit', 2:'gear', 3:'download', 4:'trash'} # just for fun, can be any marker or None
4 slides.pause(isolate=True) # isolate to split from previous content
5 slides.write(slides.group(boxes,
6     points=True, snapshots=True,
7     marker=lambda idx:f"fa['red']`{icons[idx]}`",
8     css_class='anim-group anim-slide-left', # per item animation via parent group
9 )) # snapshots reveal rows in isolation
```



4

Frames with

`pause.iter()` and 2x2 grid of boxes

Python

```
1 slides.write('# Frames with \n#### code`pause.iter()` and 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 for ws, cols in slides.pause.iter(zip(widths,objs), isolate=True): # isolate to split from previous content
6     slides.write(*cols, widths=ws, css_class='anim-group anim-wipe-right')
```

Frames with

`pause.iter()` and 2x2 grid of boxes

Python

```
1 slides.write('# Frames with \n#### code`pause.iter()` and 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 for ws, cols in slides.pause.iter(zip(widths,objs), isolate=True): # isolate to split from previous content
6     slides.write(*cols, widths=ws, css_class='anim-group anim-wipe-right')
```

1

Frames with

`pause.iter()` and 2x2 grid of boxes

Python

```
1 slides.write('# Frames with \n#### code`pause.iter()` and 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 for ws, cols in slides.pause.iter(zip(widths,objs), isolate=True): # isolate to split from previous content
6     slides.write(*cols, widths=ws, css_class='anim-group anim-wipe-right')
```

1

2

Frames with

`pause.iter()` and 2x2 grid of boxes

Python

```
1 slides.write('# Frames with \n#### code`pause.iter()` and 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 for ws, cols in slides.pause.iter(zip(widths,objs), isolate=True): # isolate to split from previous content
6     slides.write(*cols, widths=ws, css_class='anim-group anim-wipe-right')
```

1

2

3

Frames with

`pause.iter()` and 2x2 grid of boxes

Python

```
1 slides.write('# Frames with \n#### code`pause.iter()` and 2x2 grid of boxes')
2 s.get_source().focus(range(2,7)).display()
3 objs = [boxes[:2],boxes[2:]]
4 widths = [(1,3),(3,2)]
5 for ws, cols in slides.pause.iter(zip(widths,objs), isolate=True): # isolate to split from previous content
6     slides.write(*cols, widths=ws, css_class='anim-group anim-wipe-right')
```

1

2

3

4

[← Skip Previous Frames](#)

Watching Youtube Video?

Want to do some drawing instead? Click on button on the right!



IPySlides-Demo

Abdul Saboor



Watch on



IPySlides | From a Hobby to a Soli

Abdul Saboor



Watch on

[≡ Show Code](#)

Blocks with CSS classes

Table

h1	h2	h3
d1	d2	d3
r1	r2	r3

Widgets



Click to do nothing

☒ Select to do nothing

A rich content table

h1	h2	h3
)	2	3
3	<code>import numpy as np</code>	5

[≡ Show Code](#)

LAT_{EX} in Slides

Use $\$ \$$ or $\$ \$ \$ \$$ to display latex in Markdown, or embed images of equations LAT_{EX} needs time to load, so keeping it in view until it loads would help.



Tip

Variable formatting alongwith $LAT_{EX} \text{\%{var}} \rightarrow$ 'I was a variable' is seamless.

$LAT_E X$ in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.



Tip

Variable formatting alongwith $LAT_E X$ `%{var}` $\rightarrow$ 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

$LAT_E X$ in Slides

Use $\$$ $\$$ or $\$ \$$ $\$ \$$ to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.



Tip

Variable formatting alongwith $LAT_E X$ `\%{var}` $\rightarrow$ 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

$$ax^2 + bx + c = 0$$

$LATEX$ in Slides

Use `$ $` or `$$ $$` to display latex in Markdown, or embed images of equations $LATEX$ needs time to load, so keeping it in view until it loads would help.



Tip

Variable formatting alongwith $LATEX$ `\%{var}` $\rightarrow$ 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

Hello 🙋 I am pinned here!

$$ax^2 + bx + c = 0$$

$LAT_E X$ in Slides

Use $\$$ $\$$ or $\$ \$$ $\$ \$$ to display latex in Markdown, or embed images of equations $LAT_E X$ needs time to load, so keeping it in view until it loads would help.



Tip

Variable formatting alongwith $LAT_E X$ `\%{var}` $\rightarrow$ 'I was a variable' is seamless.

$$\int_0^1 \frac{1}{1-x^2} dx$$

Hello 🙋 I am pinned here!

$$ax^2 + bx + c = 0$$

≡ Show Code

Serialize Custom Objects to HTML

This is useful for displaying user defined/third party objects in slides

0

1

2

3

4

5

6

7

8

9

Python

```
1 slides.write('## Serialize Custom Objects to HTML\nThis is useful for displaying user defined/third party objects')
2 with slides.suppress_stdout(): # suppress stdout from register function below
3     @slides.serializer.register(int)
4     def colorize(obj):
5         color = 'red' if obj % 2 == 0 else 'green'
6         return f'<span style="color:{color};">{obj}</span>'
7     slides.write(*range(10))
8
9 some_slide.get_source().display()
```

This is all code to generate slides

Python

```
1 def demo(self):
2     "Demo slides with a variety of content."
3     from ..demo import demo_slides
4     demo_slides(self) # Do not return anything
```

e:\development\ipyslides\ipyslides_demo.py

```
1 # Author: Abdul Saboor
2 # This demonstrates that you can generate slides from a .py file too, which you can import in notebook.
3
4 def demo_slides(slides):
5     raw_source = slides.code.cast(__file__).raw
6     N = raw_source.count('.build') + raw_source.count('\n---')
7     slides.clear(keep = N) # Clear previous content
8     slides.create(range(N)) # Create slides first, this is faster
9
10    slides.settings.footer.text = slides.get_logo("1em") + 'Author: Abdul Saboor عبدالصبور'
11    slides.set_citations(r'''
12        @pf: This is refernce to FigureWidget using alert`cite`\`pf\`` syntax
13        @This: I was cited for no reason
```

Source Code

[Markdown: Slide 0](#)

```
1  ```md-src.collapsed
2  # Creating Slides
3  ::: align-center width=50%
4      alert`Abdul Saboor``1`, Unknown Author``2`
5      center`//today``//`
6      ::: align-left text-box
7          ``1`My University is somewhere in the middle of nowhere
8          ``2`Their University is somewhere in the middle of nowhere
9
10 ::: display align-center
11     vspace`2`Right click (or click on footer) to open context menu and click on fa`info` icon for i
12 ```
13 <md-src/>
```

[Markdown: Slide 1](#)

```
1 section`Introduction` toc`### Contents`
```

[Markdown: Slide 2](#)

```
1 # Introduction
```